计算机体系结构

07b. 数据通路

李建华
计算机与信息学院

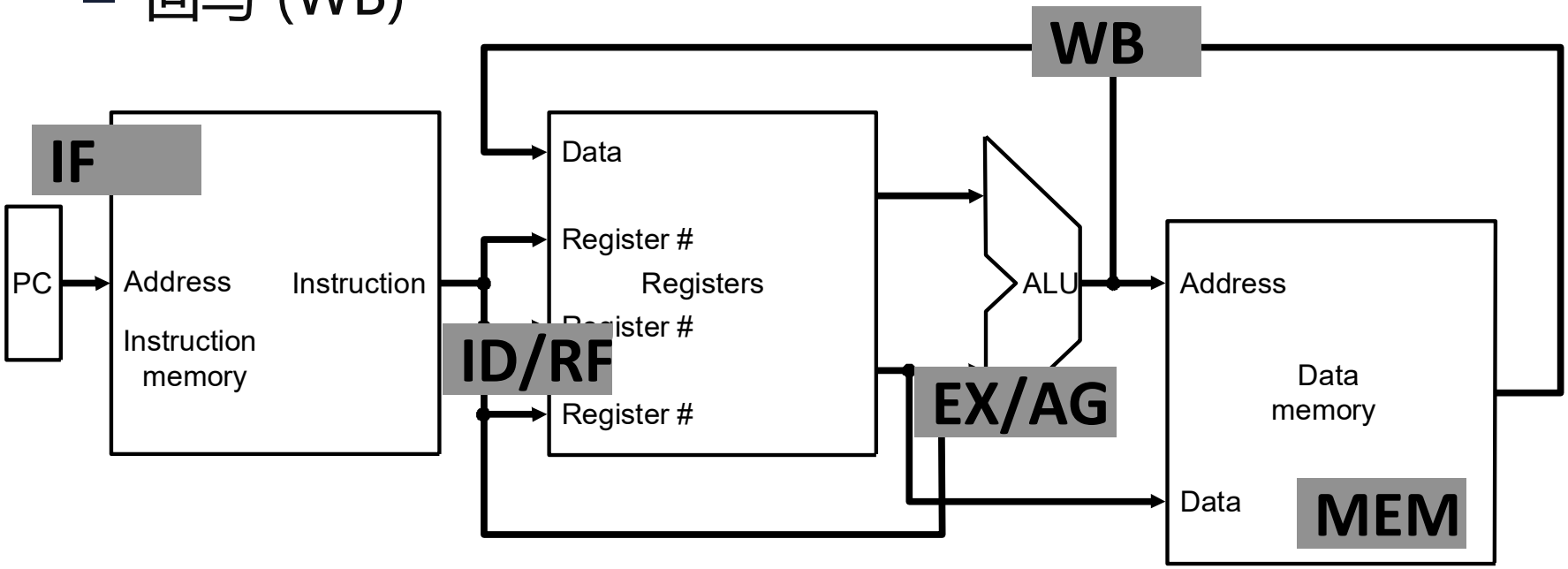
本讲提纲

- MIPS指令处理步骤
- 算术逻辑指令的数据通路
- 访存指令的数据通路
- 控制流指令数据通路
- MIPS单周期控制逻辑
- MIPS单周期性能分析

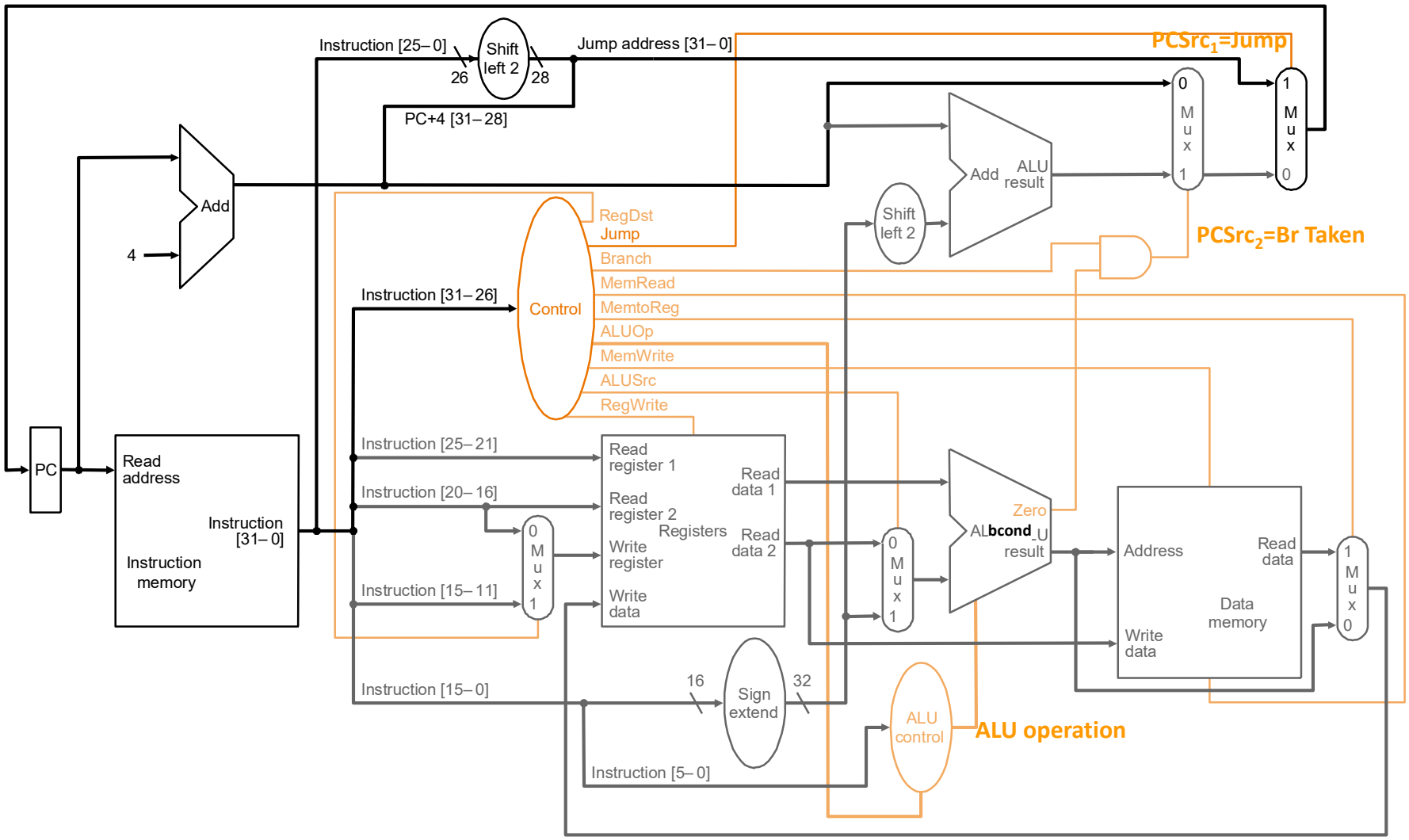
这是补充的内容，主要的目的是：
为大家更好地开展《系统硬件综合设计》做准备工作。

MIPS指令的处理步骤

- 主要步骤：
  - 取指 (IF)
  - 译码 & 取寄存器操作数 (ID&RF)
  - 执行/计算访存地址 (EX/AG)
  - 访存 (MEM)
  - 回写 (WB)

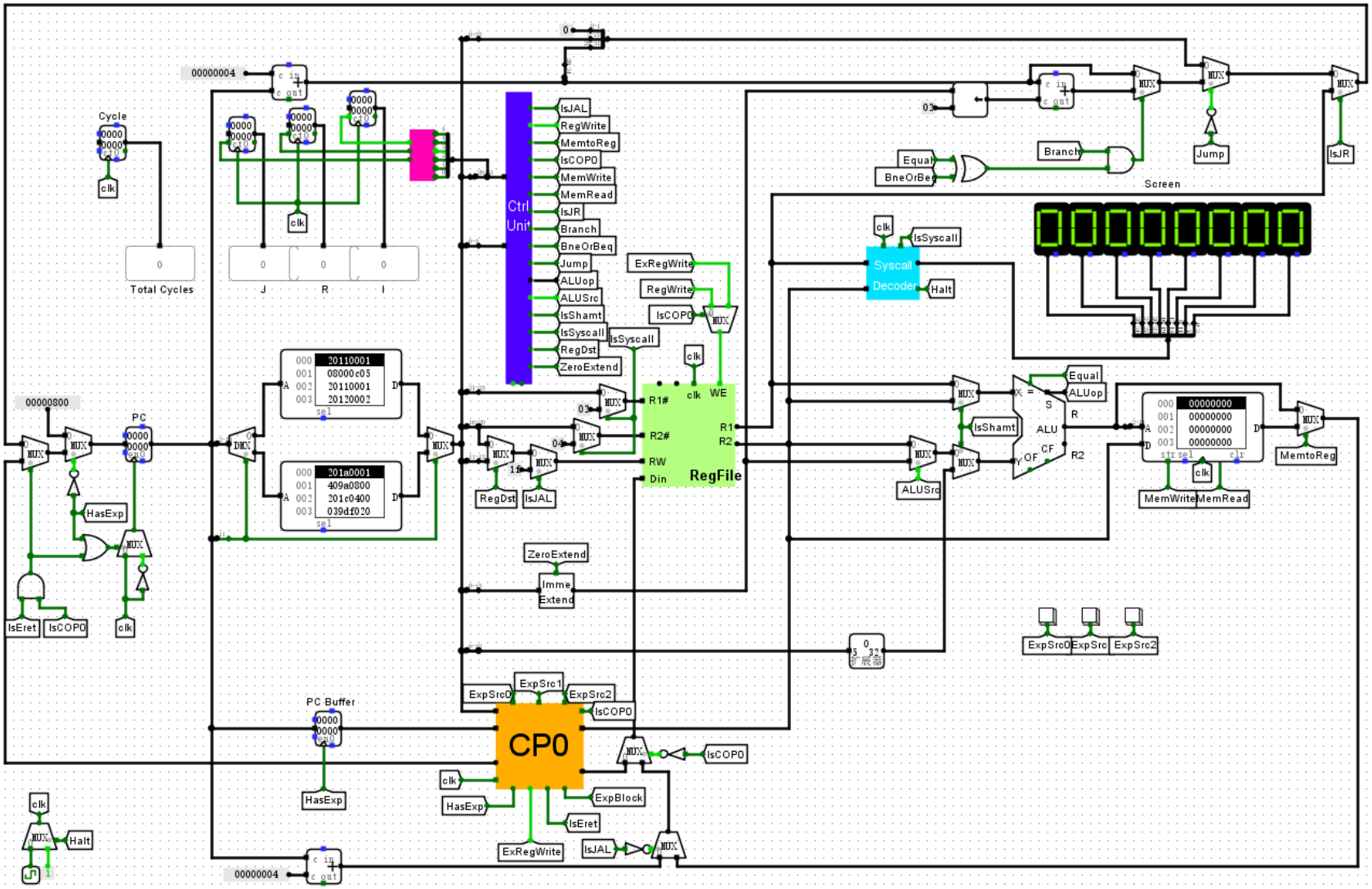


相对完整的单周期数据通路



JAL, JR, JALR omitted

用logisim搭建的单周期CPU



本讲提纲

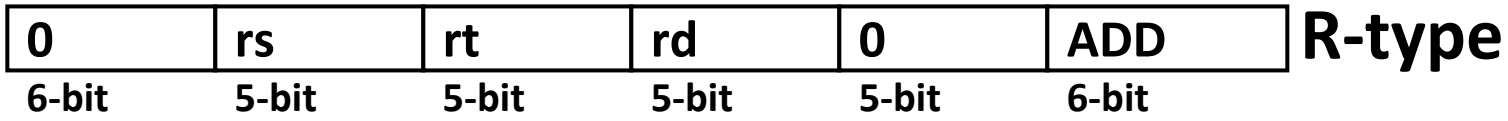
- MIPS指令处理步骤
- 算术逻辑指令的数据通路
- 访存指令的数据通路
- 控制流指令数据通路
- MIPS单周期控制逻辑
- MIPS单周期性能分析

R-Type ALU 指令

- 汇编 (e.g., register-register signed addition)

  ADD rd_{reg} rs_{reg} rt_{reg}

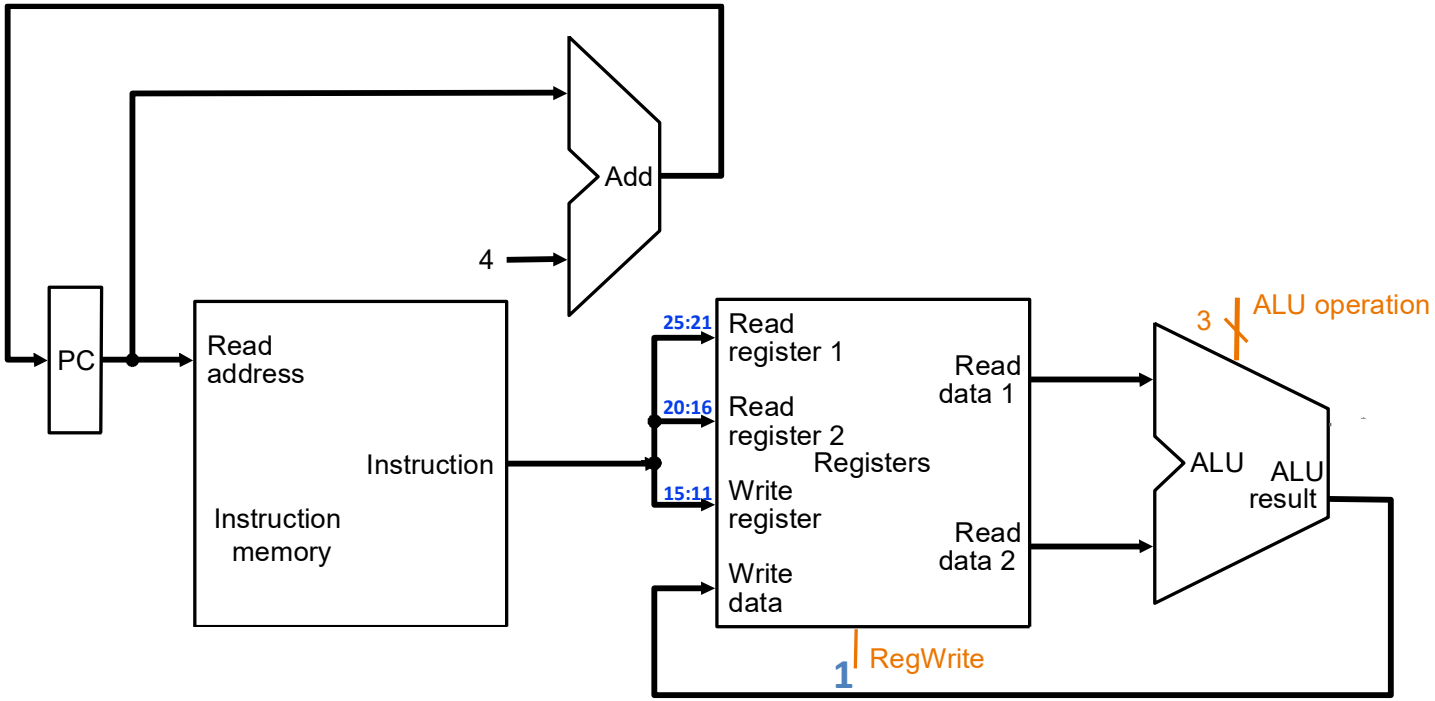
- 机器编码



- 语义

  if MEM[PC] == ADD rd rs rt
  GPR[rd] ← GPR[rs] + GPR[rt]
  PC ← PC + 4

R-Type ALU 数据通路



IF	ID	EX	MEM	WB

if MEM[PC] == ADD rd rs rt

GPR[rd] ← GPR[rs] + GPR[rt]

PC ← PC + 4

状态更新组合逻辑

I-Type ALU 指令

- 汇编 (e.g., register-imm signed additions)

  ADDI rt_{reg} rs_{reg} $immediate_{16}$

- 机器编码

I-type

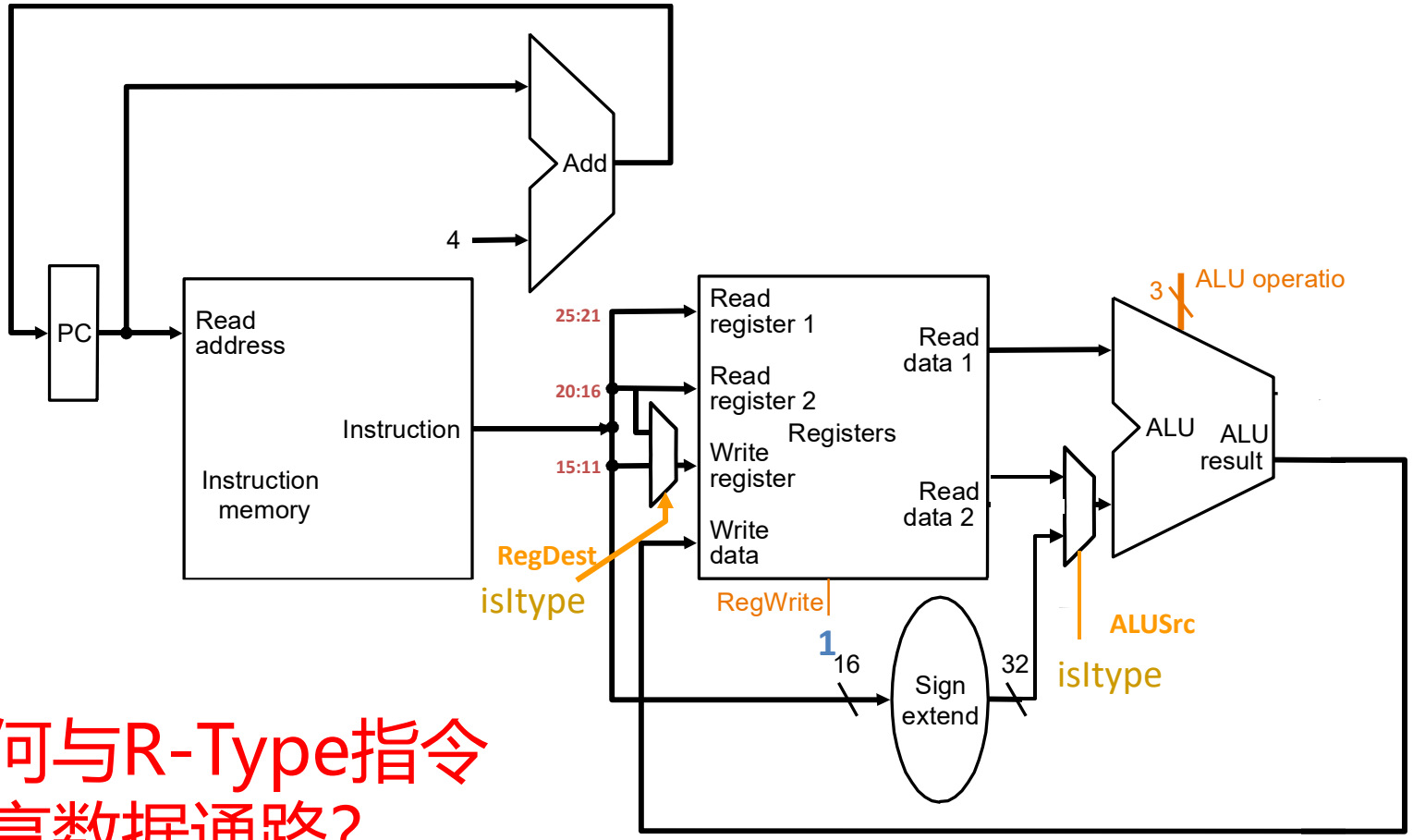
- 语义

  if MEM[PC] == ADDI rt rs immediate

  GPR[rt] ← GPR[rs] + sign-extend (immediate)

  PC ← PC + 4

R and I-Type ALU 指令的数据通路



如何与R-Type指令共享数据通路?

```
if MEM[PC] == ADDI rt rs immediate
    GPR[rt] ← GPR[rs] + sign-extend
    (immediate)
    PC ← PC + 4
```

IF	ID	EX	MEM	WB

状态更新组合逻辑

本讲提纲

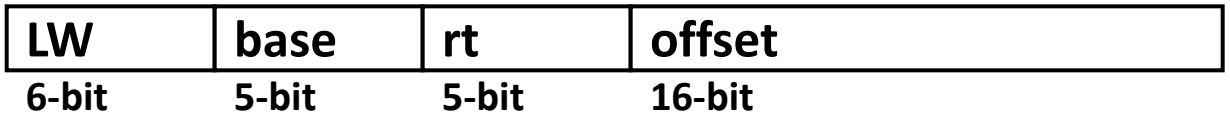
- MIPS指令处理步骤
- 算术逻辑指令数据通路
- 访存指令的数据通路
- 控制流指令数据通路
- MIPS单周期控制逻辑
- MIPS单周期性能分析

I-Type LW 指令

- 汇编 (e.g., load 4-byte word)

  LW rt_{reg} $offset_{16}$ ($base_{reg}$)

- 机器编码

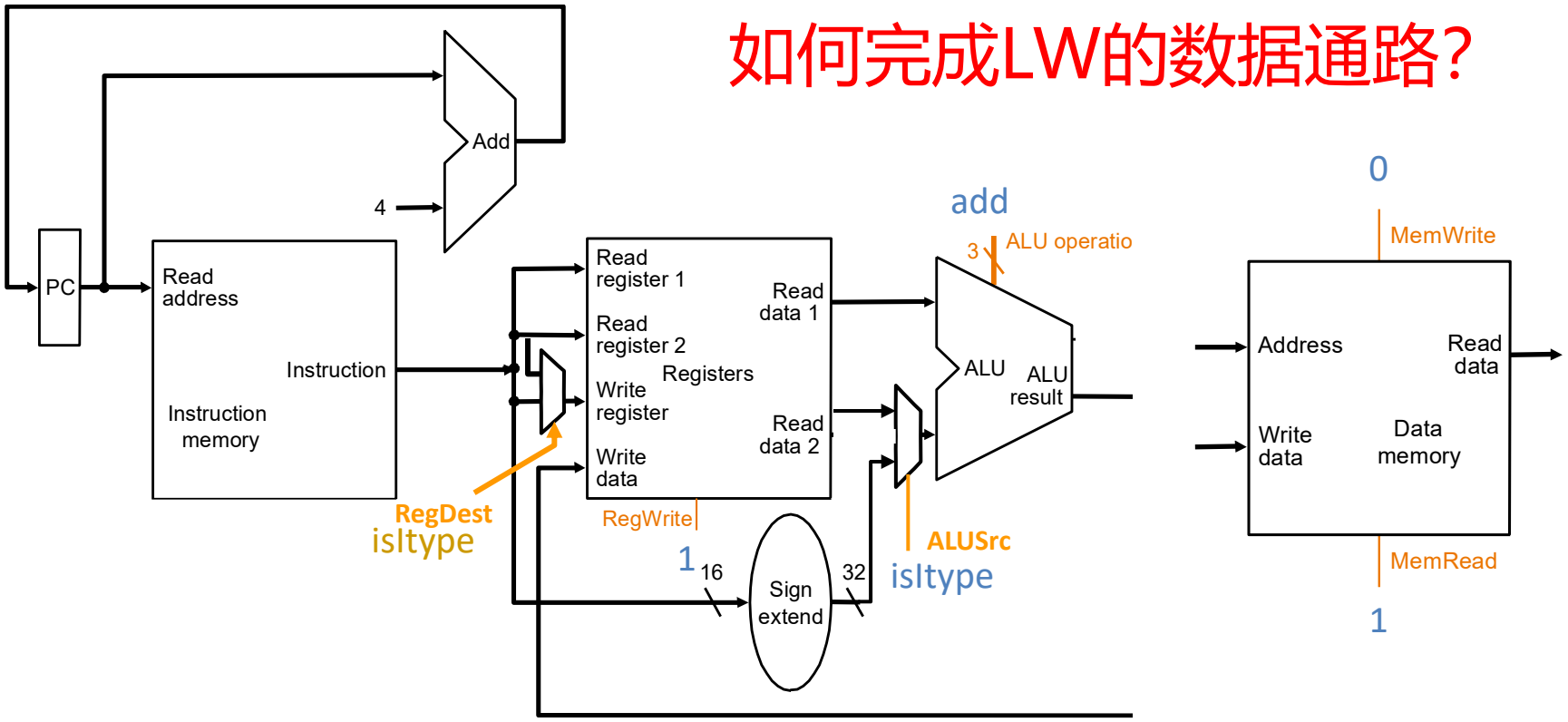


I-type

- 语义

  if MEM[PC]==LW rt $offset_{16}$ (base)
  EA = sign-extend(offset) + GPR[base]
  GPR[rt] ← MEM[translate(EA)]
  PC ← PC + 4

LW 数据通路



if MEM[PC]==LW rt offset$_{16}$ (base)

EA = sign-extend(offset) + GPR[base]

GPR[rt] ← MEM[translate(EA)]

PC ← PC + 4

IF	ID	EX	MEM	WB

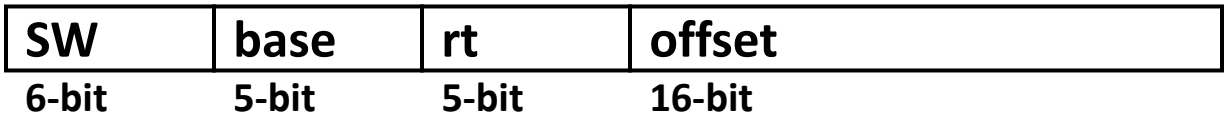
状态更新组合逻辑

I-Type SW 指令

- 汇编 (e.g., store 4-byte word)

  SW rt_{reg} $offset_{16}$ ($base_{reg}$)

- 机器编码



I-type

- 语义

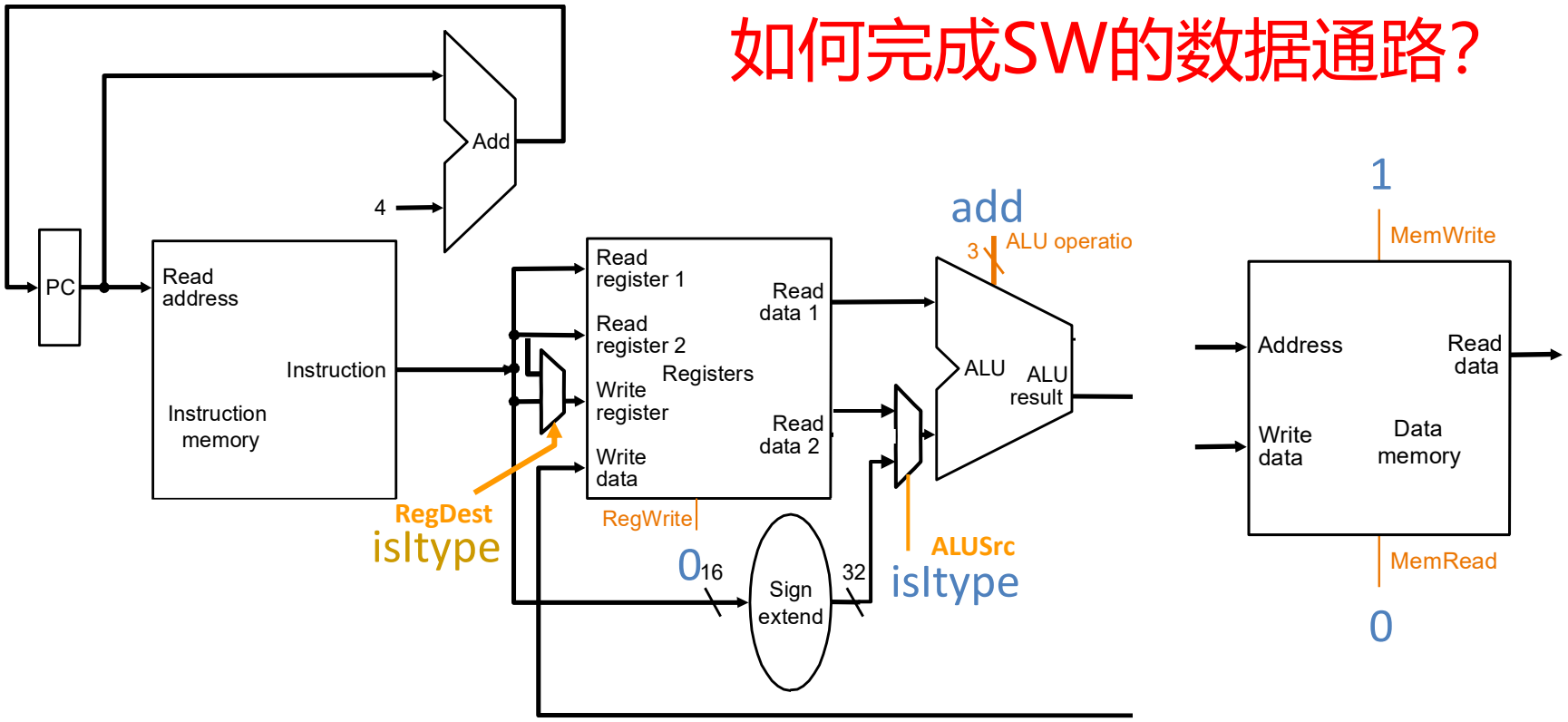
  if MEM[PC]==SW rt $offset_{16}$ (base)

  EA = sign-extend(offset) + GPR[base]

  MEM[translate(EA)] ← GPR[rt]

  PC ← PC + 4

SW 数据通路



if MEM[PC]==SW rt offset$_{16}$ (base)

EA = sign-extend(offset) + GPR[base]

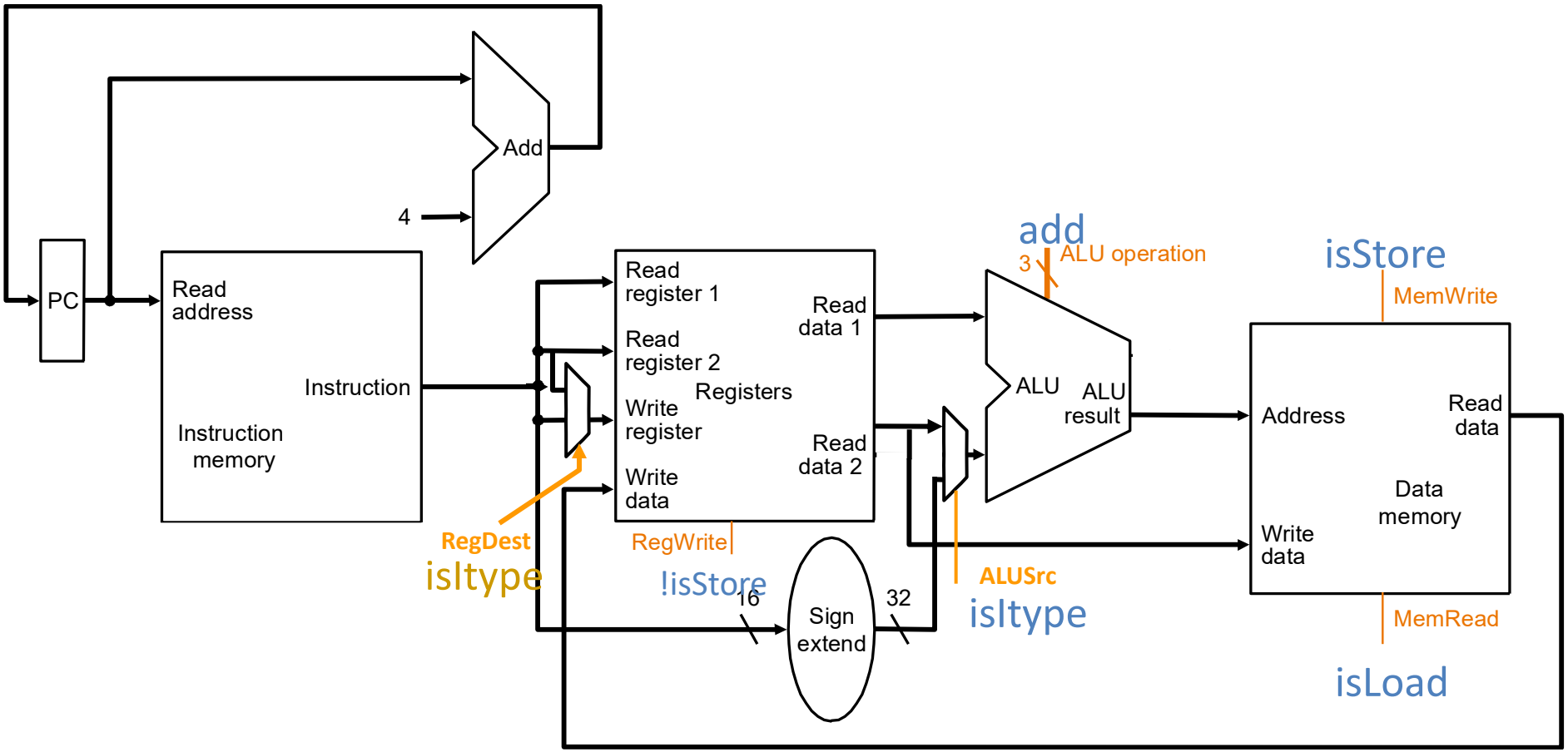
MEM[translate(EA)] ← GPR[rt]

PC ← PC + 4

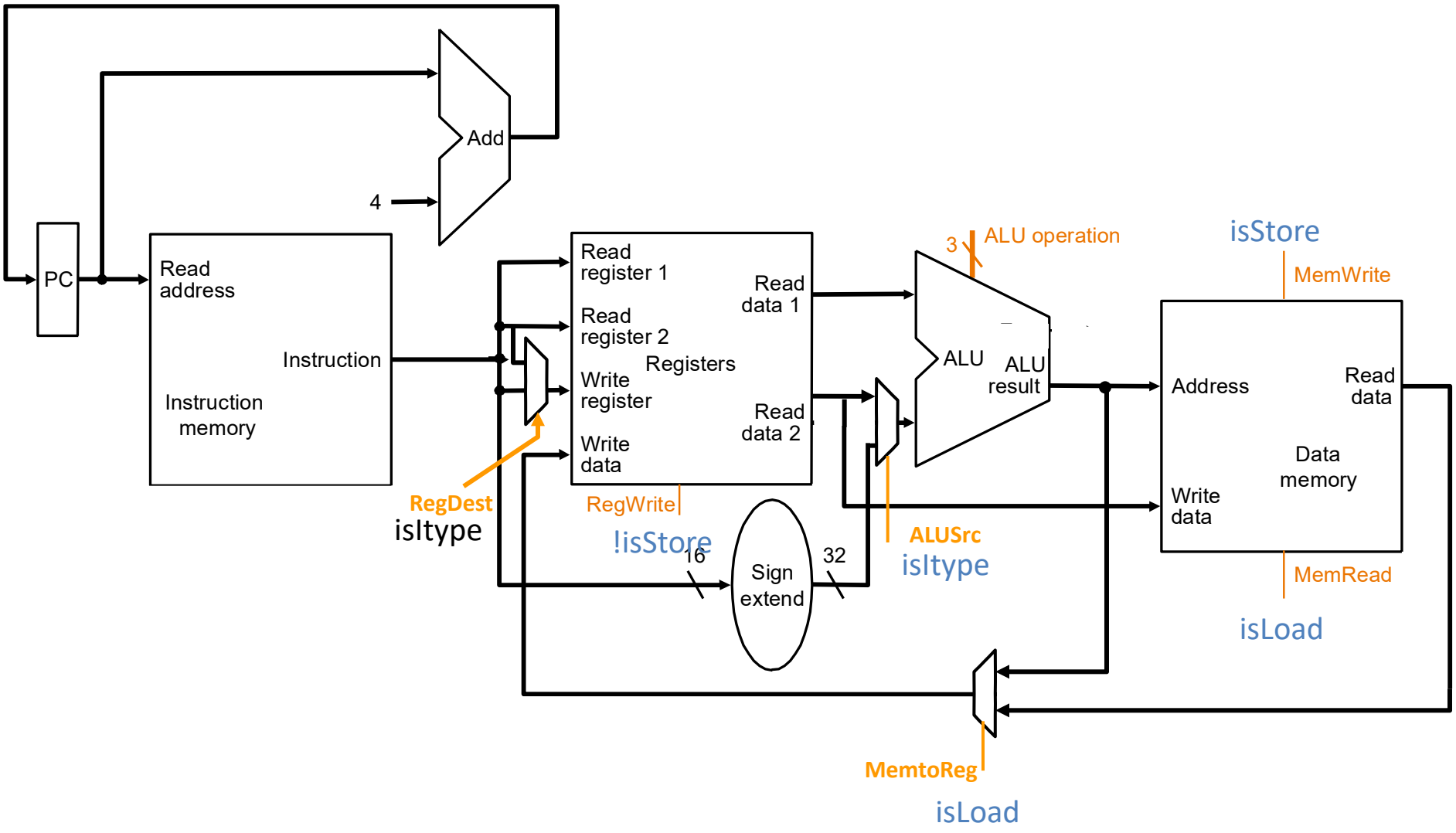
IF	ID	EX	MEM	WB

状态更新组合逻辑

Load-Store 数据通路



非控制流指令的数据通路



本讲提纲

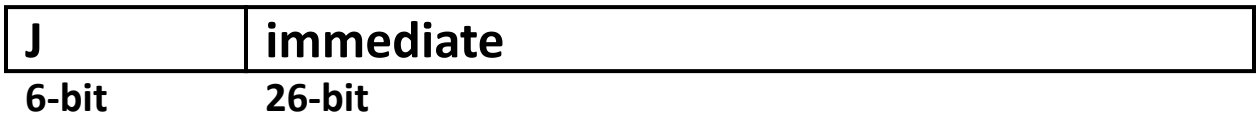
- MIPS指令处理步骤
- 算术逻辑指令数据通路
- 访存指令的数据通路
- 控制流指令数据通路
- MIPS单周期控制逻辑
- MIPS单周期性能分析

无条件跳转指令

- 汇编

  J $immediate_{26}$

- 机器编码



J-type

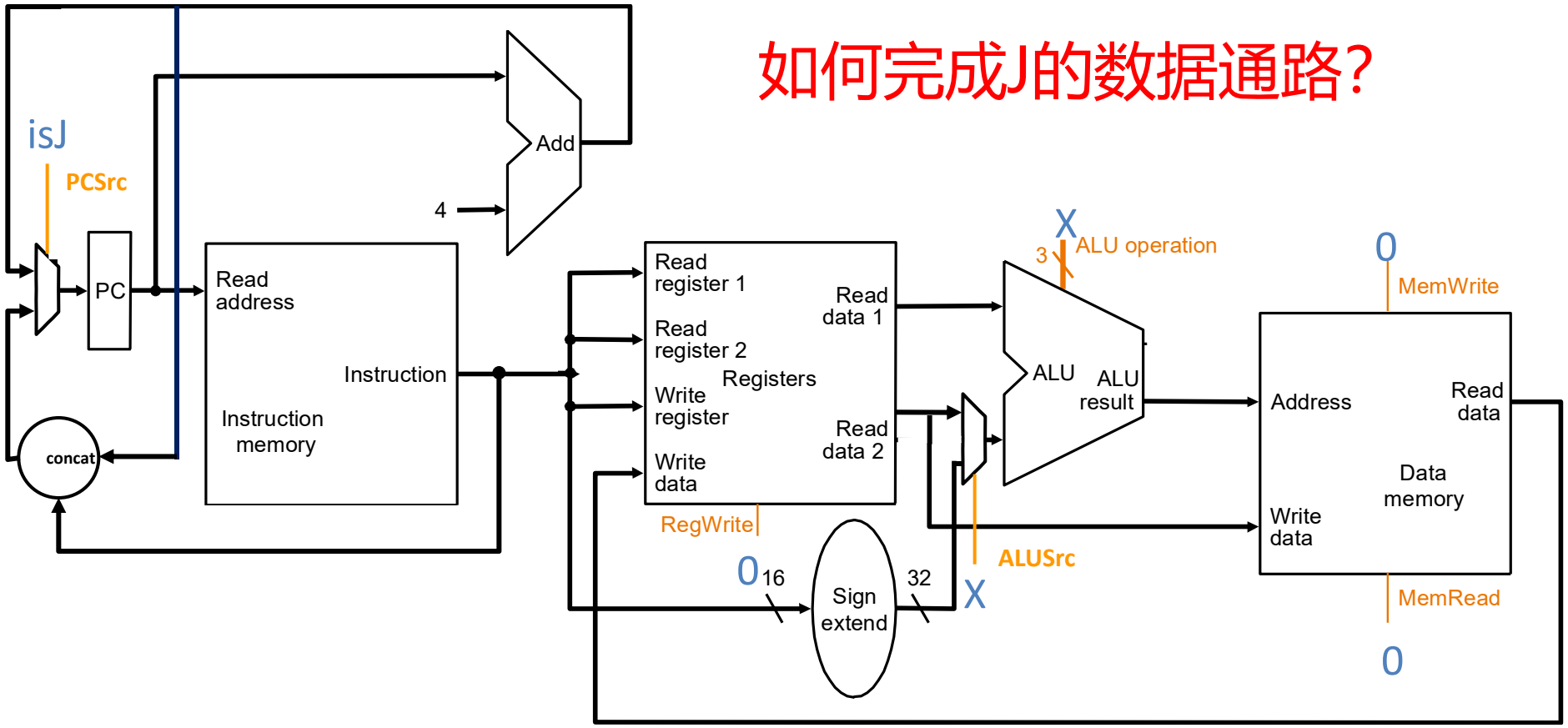
- 语义

  if MEM[PC]==J $immediate_{26}$

  target = { (PC+4)[31:28], $immediate_{26}$, 2’b00 }

  PC ← target

J 数据通路



如何完成J的数据通路?

if MEM[PC]==J immediate26

PC = { PC+4[31:28], immediate26, 2'b00 }

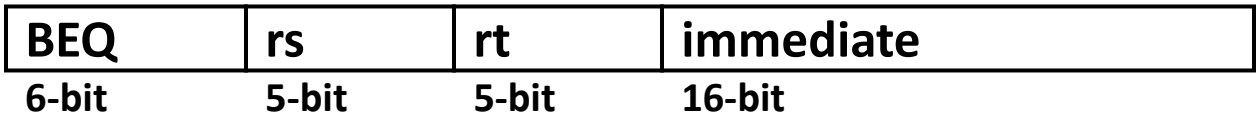
JR, JAL, JALR的数据通路?

I-Type 条件分支指令

- 汇编 (e.g., branch if equal)

  BEQ rs_{reg} rt_{reg} $immediate_{16}$

- 机器编码



I-type

- 语义 (假定没有分支延迟槽，i.e. delay slot)

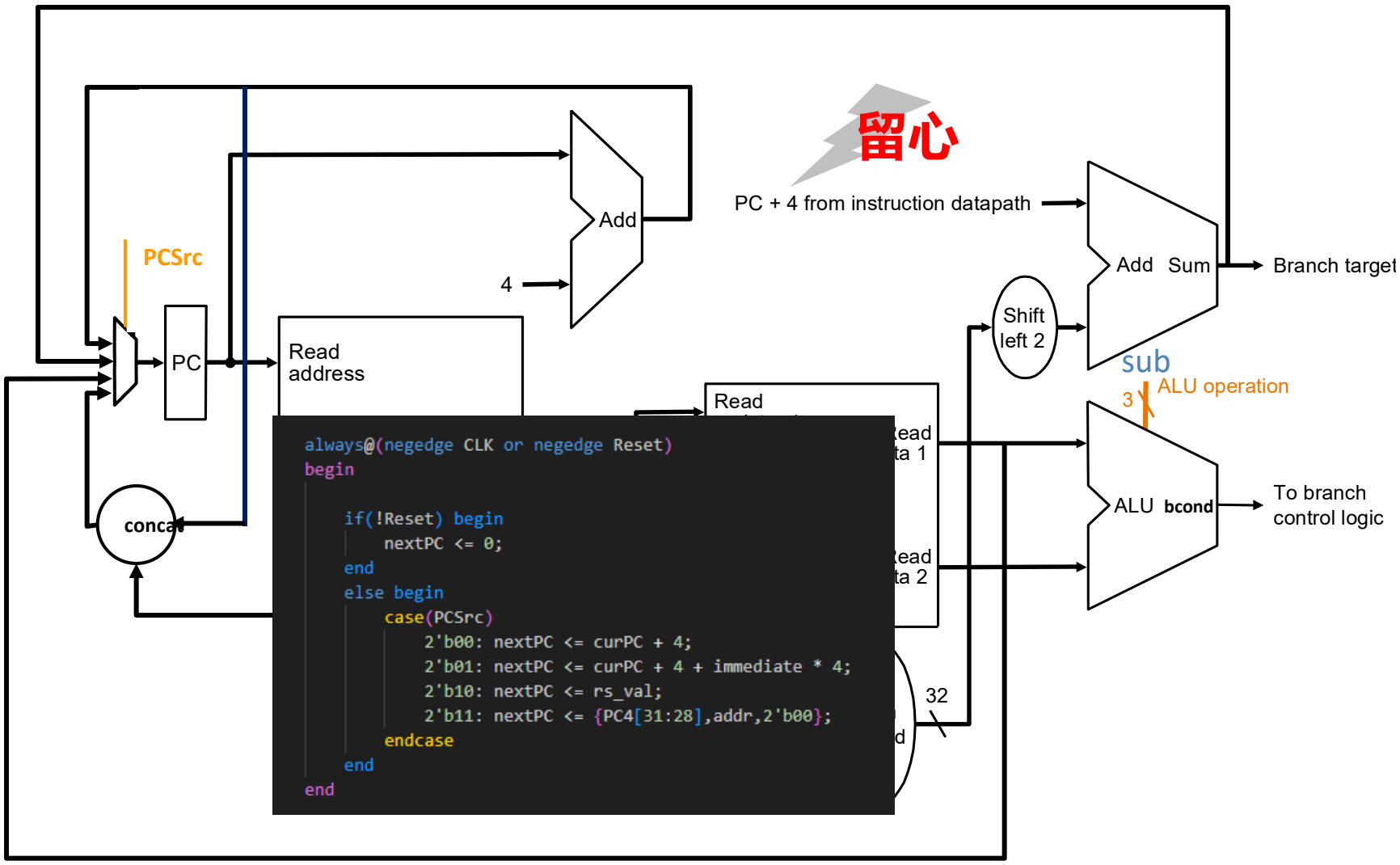
  if MEM[PC]==BEQ rs rt $immediate_{16}$

  target = PC+4 + sign-extend(immediate) x 4

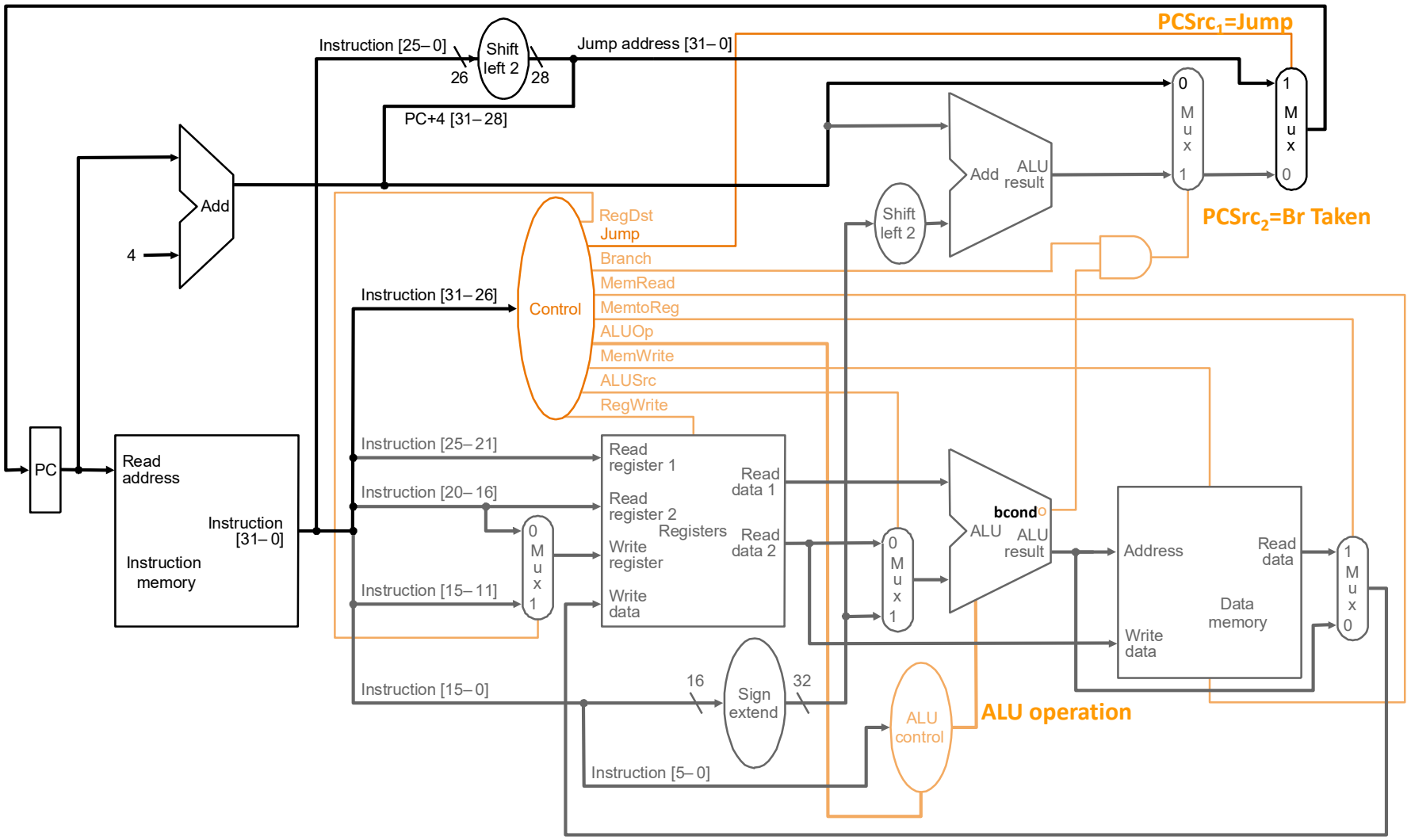
  if GPR[rs]==GPR[rt] then PC ← target

  else PC ← PC + 4

条件分支数据通路 (自己完成)



合并版本



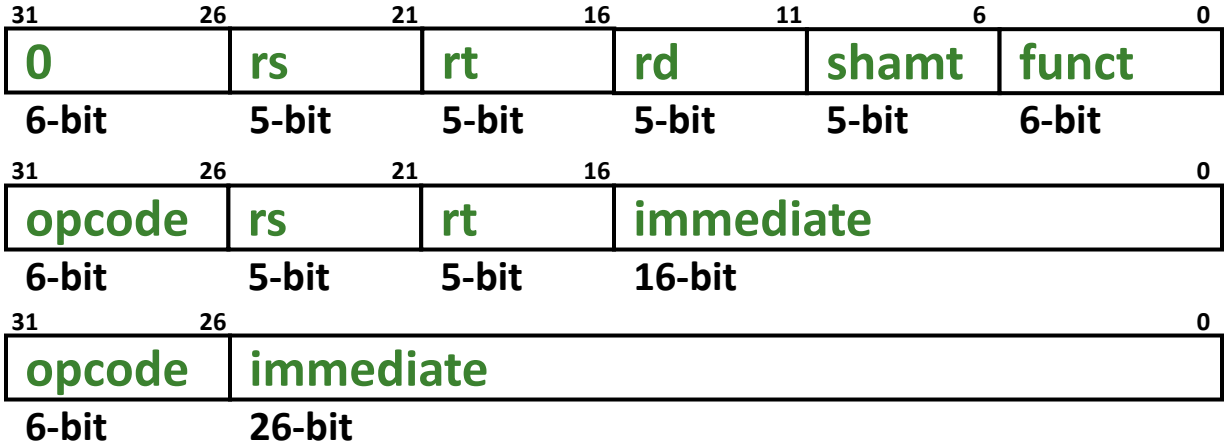
JAL, JR, JALR omitted

本讲提纲

- MIPS指令处理步骤
- 算术逻辑指令数据通路
- 访存指令的数据通路
- 控制流指令数据通路
- MIPS单周期控制逻辑
- MIPS单周期性能分析

单周期的硬布线控制

- 控制信号由Inst=MEM[PC]的组合逻辑产生



- 考虑:
  - 所有的R-type 和 I-type ALU 指令
  - LW 和 SW
  - BEQ, BNE, BLEZ, BGTZ
  - J, JR, JAL, JALR

单个比特控制信号

	件不成立	条件成立	条件
RegDest	GPR write select according to rt, i.e.,	GPR write select according to rd, i.e., inst[15:11]	opcode==0
ALUSrc	ALU input from 2nd GPR read port	2nd ALU input from sign-extended 16-bit immediate	(opcode!=0) && (opcode!=BEQ) && (opcode!=BNE)
MemtoReg	ALU result to GPR write port	steer memory load to GPR wr. port	opcode==LW
RegWrite		GPR write enabled	(opcode!=SW) && (opcode!=Bxx) && (opcode!=J) && (opcode!=JR))

目标寄存器

ALU的第二个源

从Mem到寄存器

是否要写寄存器

JAL and JALR 需要额外的RegDest and MemtoReg 选项

单个比特控制信号

		条件成立	条件
MemRead	Memory read disabled	Memory read port return load value	opcode==LW
MemWrite	Memory write disabled	Memory write enabled	opcode==SW
$PCSrc_1$	next PC = PC + 4	next PC is based on 26-bit immediate jump target	(opcode==J) || (opcode==JAL)
$PCSrc_2$	next PC = PC + 4	next PC is based on 16-bit immediate branch target	(opcode==Bxx) && "bcond is satisfied"

是否需要读Mem?

是否需要写Mem?

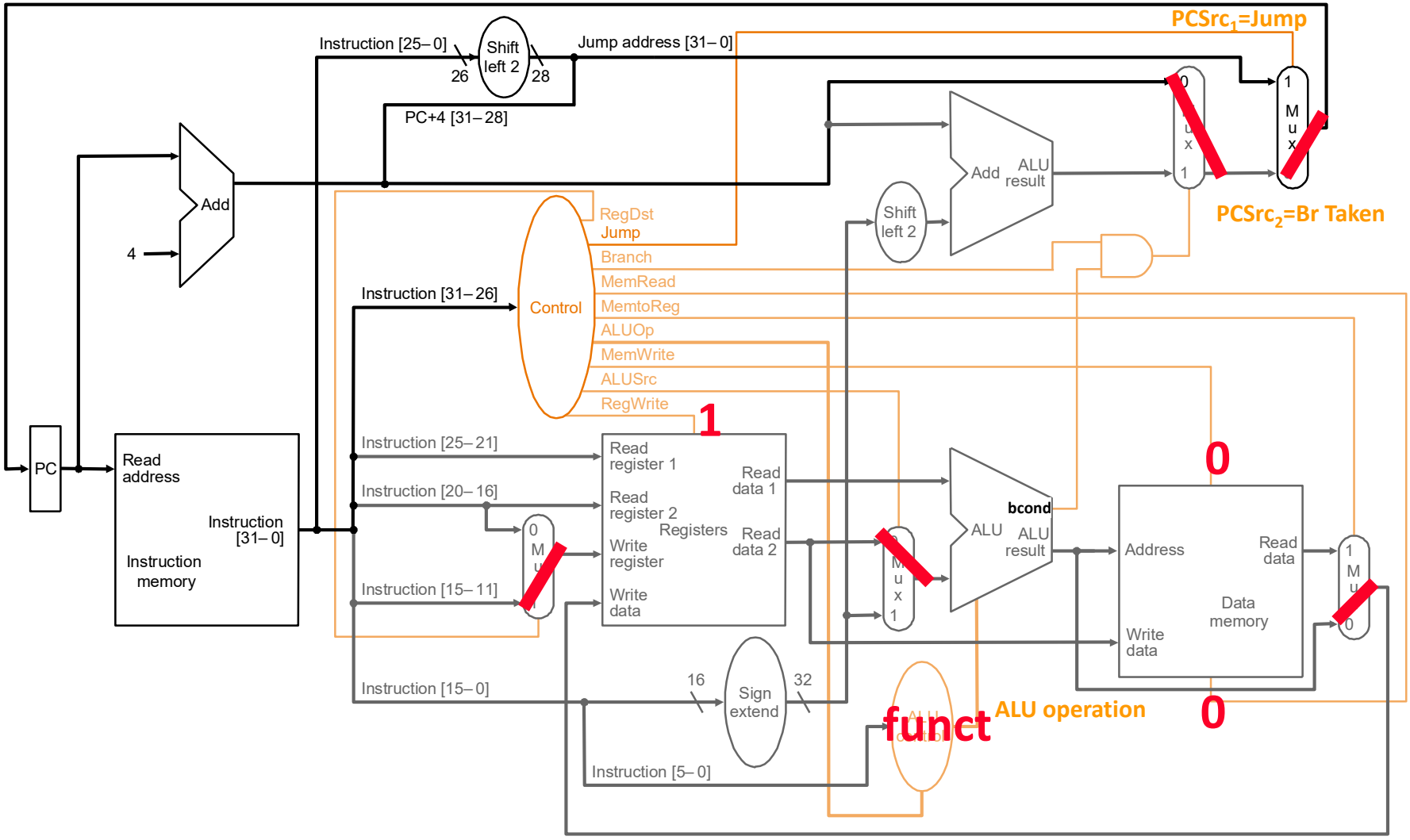
共同决定下一个PC的来源?

JR and JALR 需要额外的PCSrc 选项

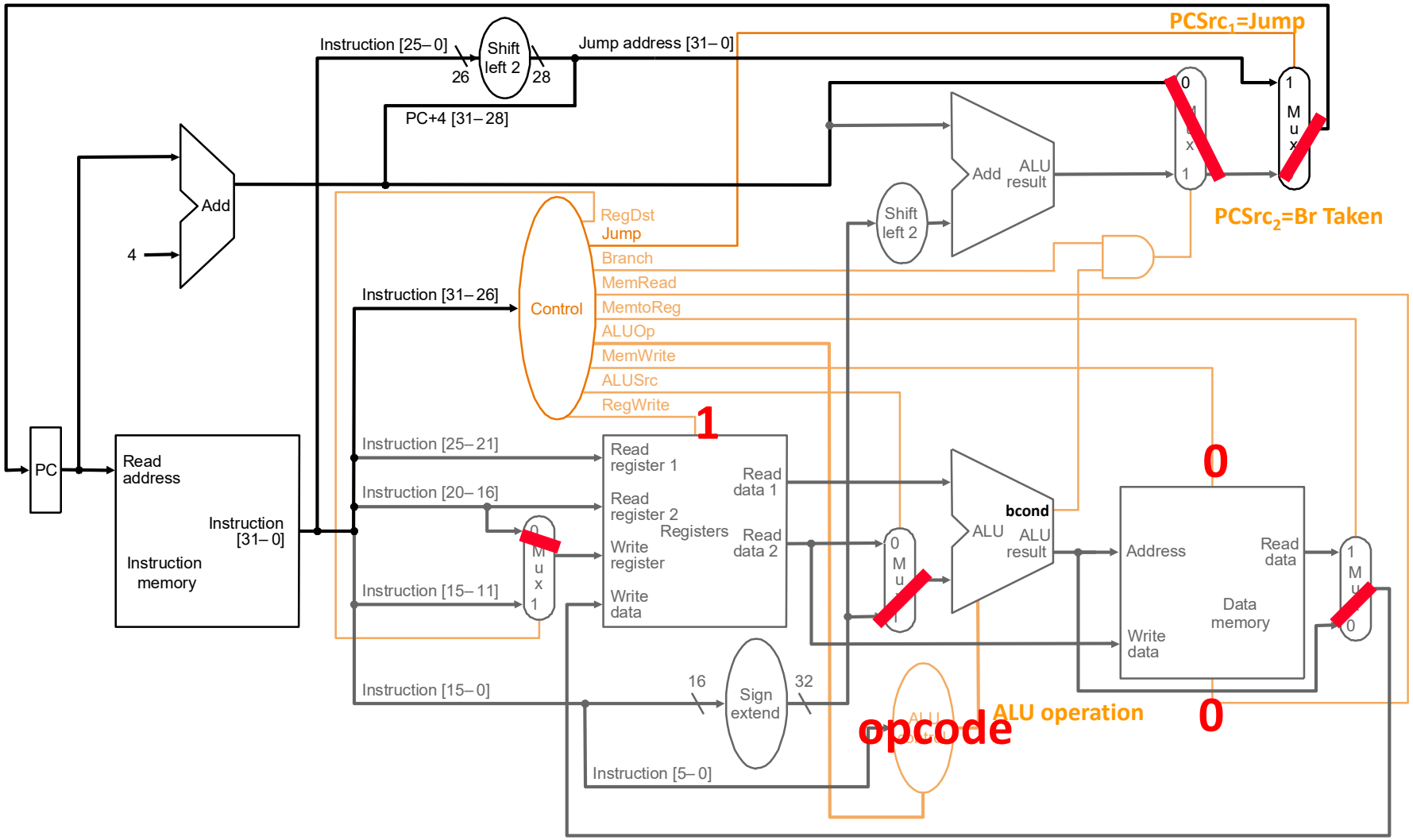
ALU 控制的实现

- case opcode
  - ‘0’ ⇒ select operation according to **funct**
  - ‘ALUi’ ⇒ selection operation according to **opcode**
  - ‘LW’ ⇒ select addition
  - ‘SW’ ⇒ select addition
  - ‘Bxx’ ⇒ select bcond generation function
  - _ ⇒ don’t care
- 示例ALU 操作
  - ADD, SUB, AND, OR, XOR, NOR, etc.
  - bcond on equal, not equal, LE zero, GT zero, etc.

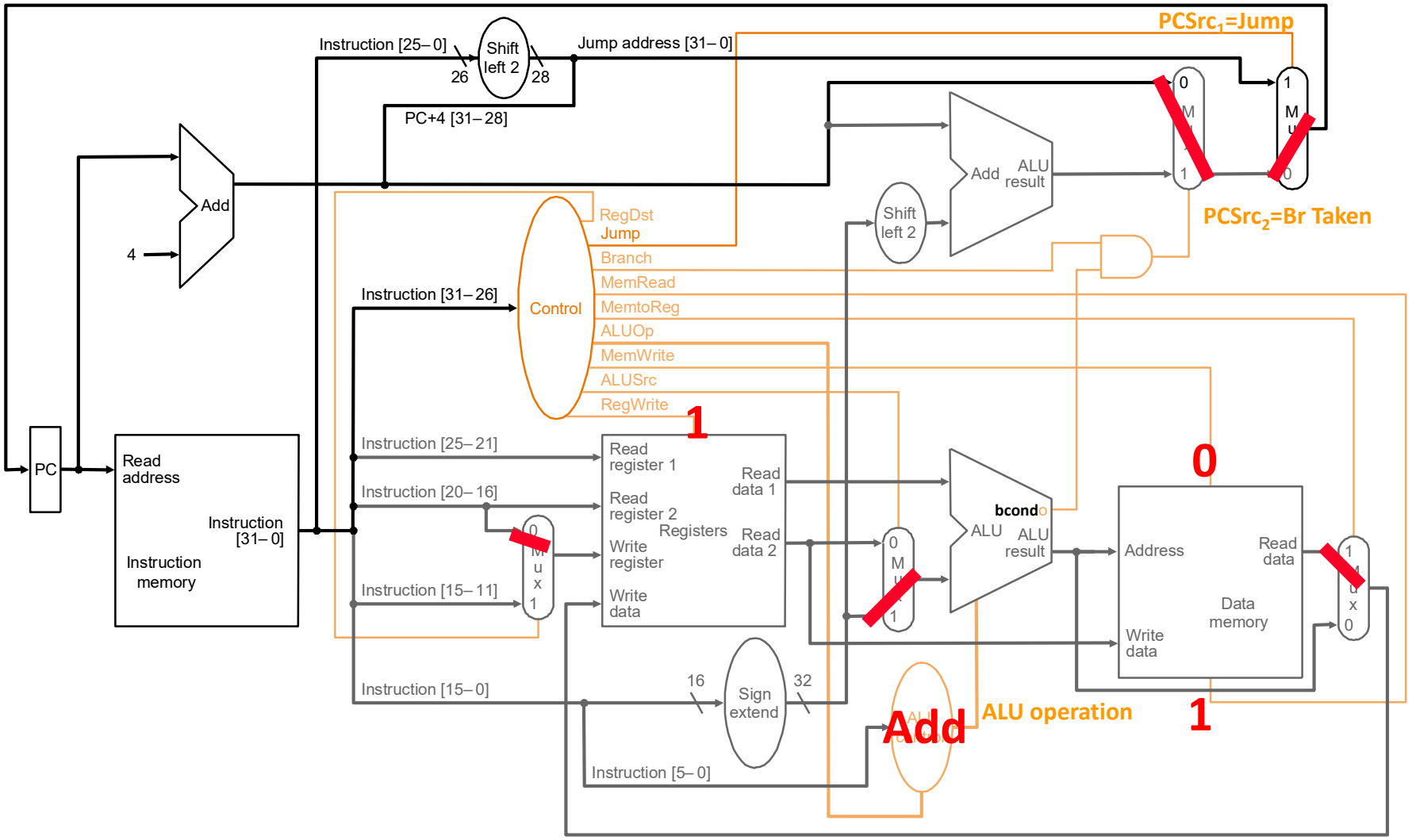
R-Type ALU


PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
M u x
0
1
Add
ALU result
Shift left 2
PCSrc2=Br Taken
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [31–26]
1
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
ALU
bcond
ALU result
0
Address
Read data
Data memory
Write data
ALU operation
funct

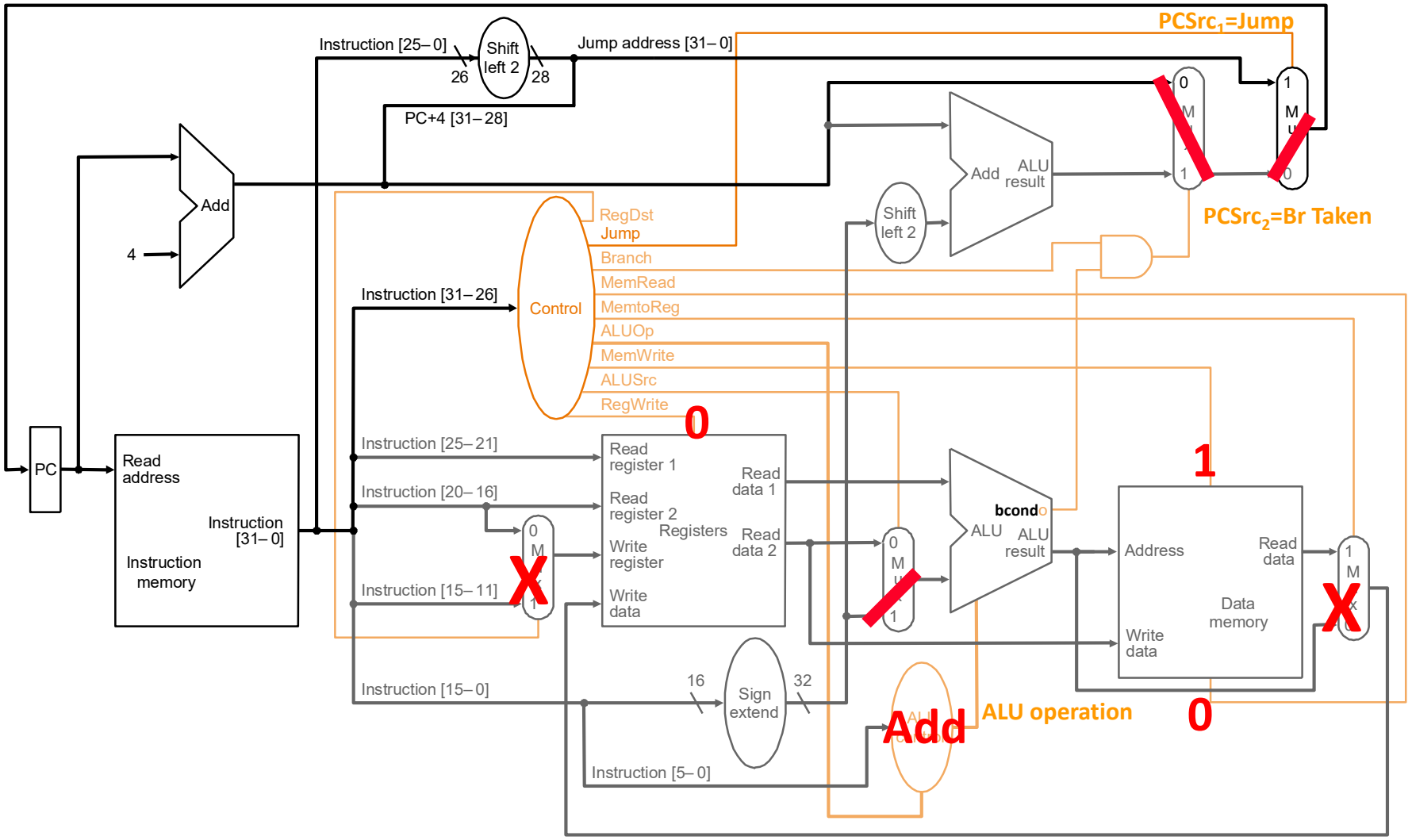
I-Type ALU


PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
Add
ALU result
Shift left 2
0
M u x
1
1
M u x
PCSrc2=Br Taken
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [31–26]
1
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
0
M u x
1
16
Sign extend
32
ALU
bcond
ALU result
0
Address
Read data
Data memory
Write data
1
M u x
0
0
opcode
ALU operation

LW

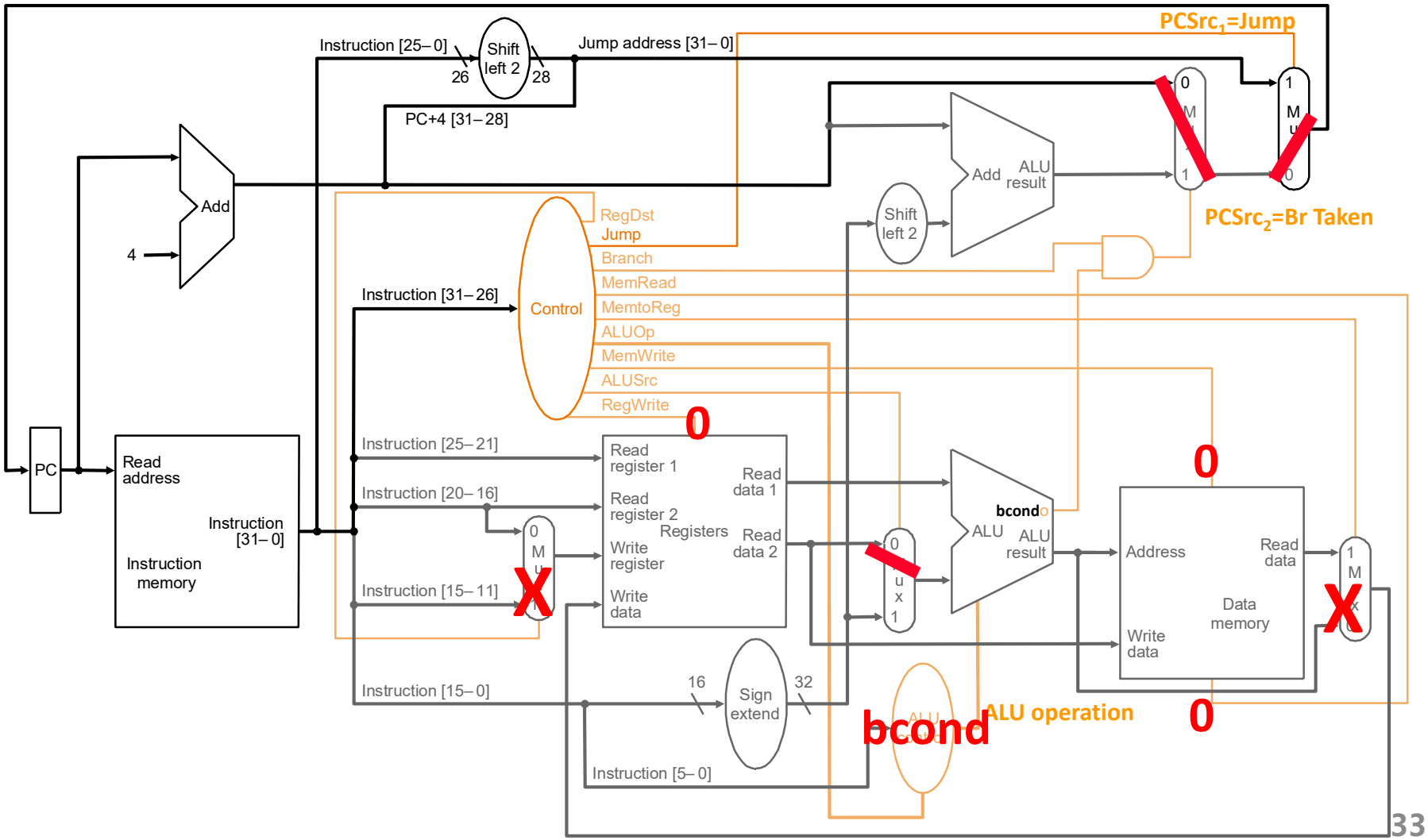

PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
M u x
0
1
PCSrc2=Br Taken
Add
ALU result
Shift left 2
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Instruction [31–26]
Control
1
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
bcond
ALU
ALU result
Add
ALU operation
0
Address
Read data
Data memory
Write data
1

SW


PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
M u x
0
1
PCSrc2=Br Taken
Add
ALU result
Shift left 2
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Instruction [31–26]
Control
0
1
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
bcond
ALU
ALU result
Add
ALU operation
Address
Read data
Data memory
Write data
X
X
0
1

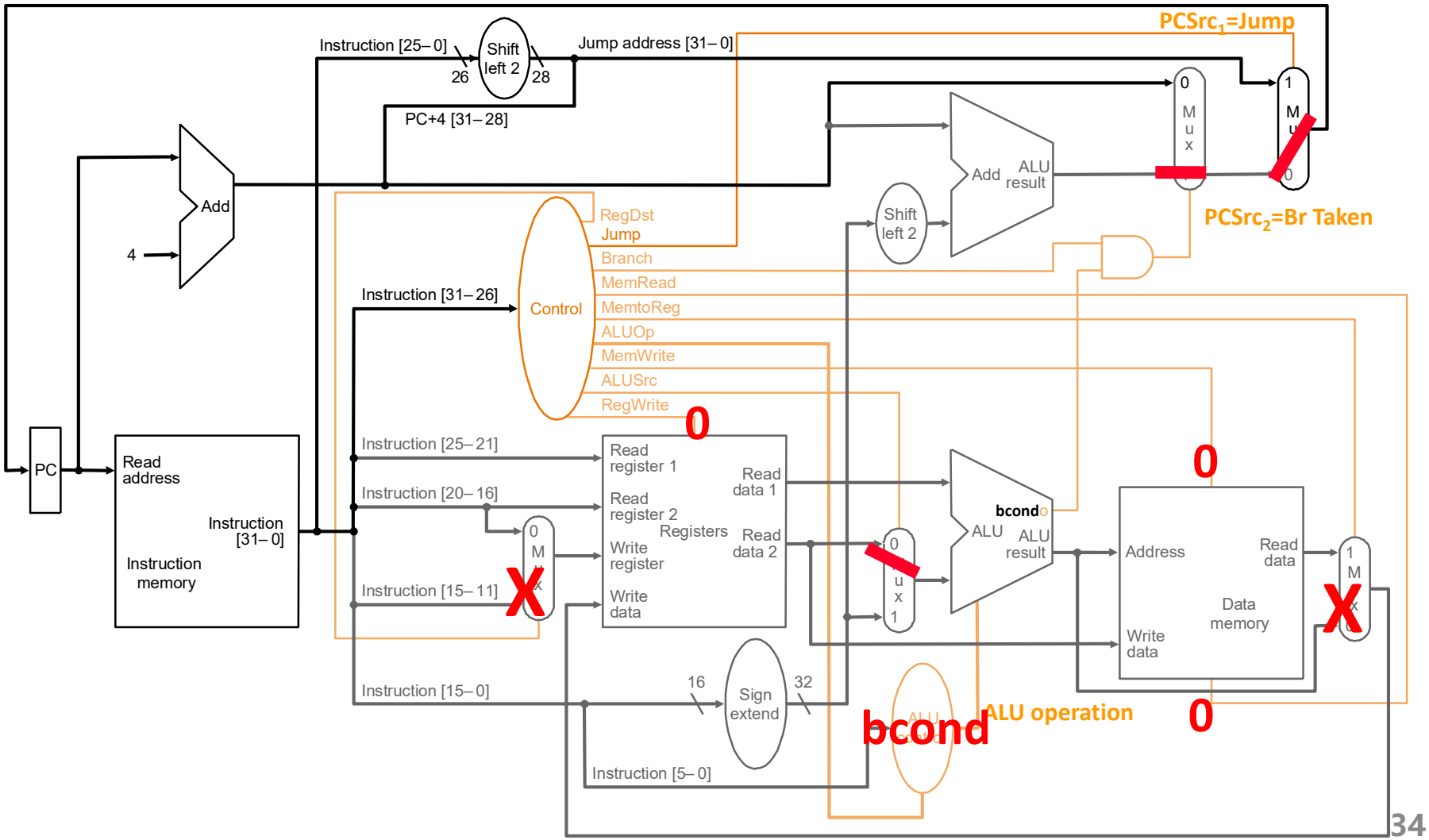
Branch (不跳转)

有些控制信号是在指令的处理过程中产生的

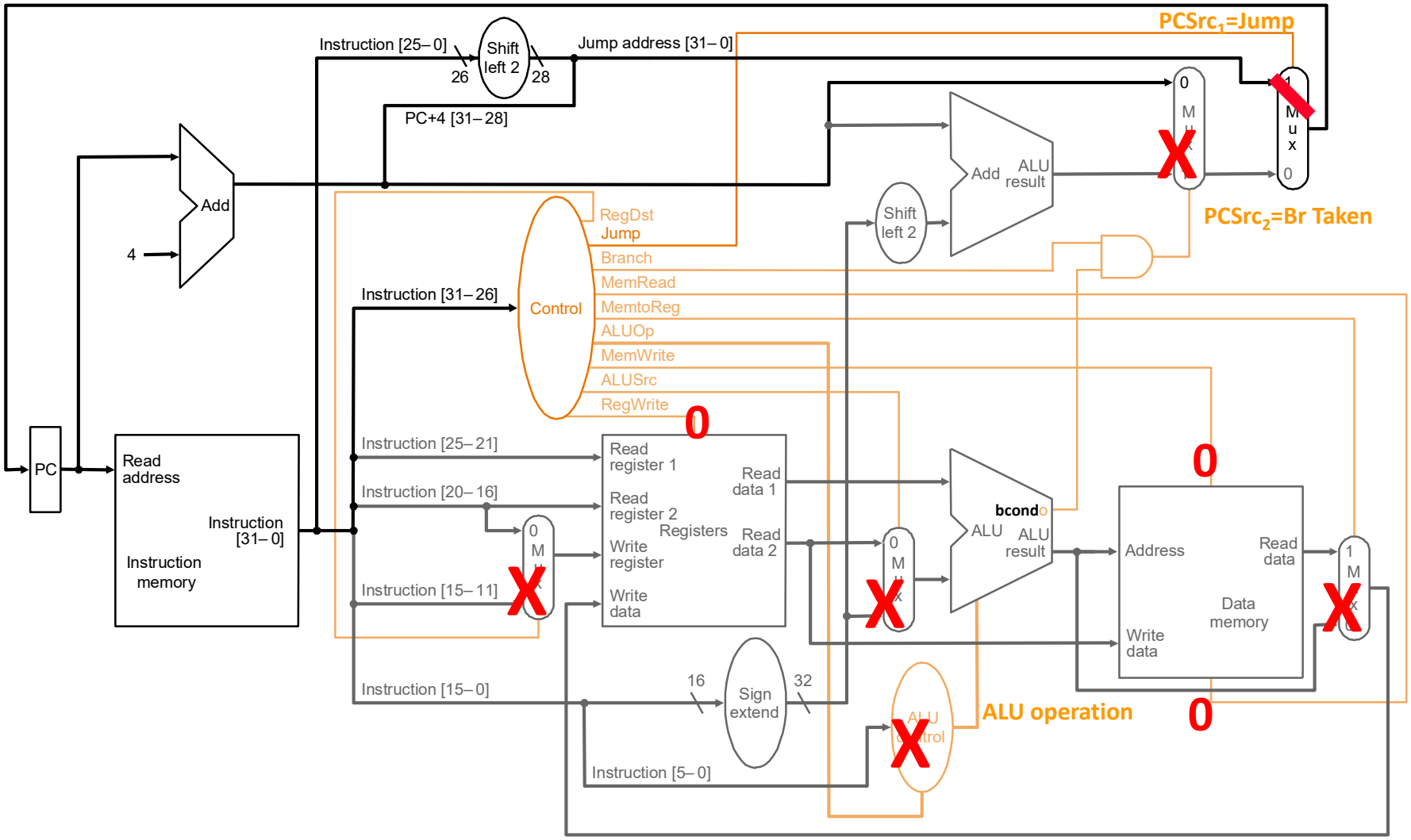


Branch (跳转)

有些控制信号是在指令的处理过程中产生的



Jump


PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
M u x
0
1
PCSrc2=Br Taken
Add
ALU result
Shift left 2
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [31–26]
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
ALU
bcond
ALU result
ALU operation
Address
Read data
Data memory
Write data
0
X

控制模块里面都有什么?

- 组合逻辑→ 硬布线控制
  - 想法: 控制信号是用组合逻辑处理指令来生成
  - 当前的CPU大多是基于这种方法实现控制...

PHOTOGRAPHS

MAURICE V. WILKES

United Kingdom – 1967

Additional Materials

Microprogramming

Microprogramming was invented by M. V. Wilkes of Cambridge University in 1951. It was a means of simplifying the control circuits of a computing system. The first machine to use the technique was the EDSAC 2 which went into service in early 1958.

In a microprogrammed computer the instructions used by the programmer are implemented by executing a sequence of primitive instructions stored in a read-only control memory. For example, a multiply instruction might be implemented using a sequence of simple shift and add instructions. The process is invisible to the user.

The microprogramming principle went commercially mainstream in 1964 when it was adopted for the IBM System/360 family of computers. The technique enabled this series of computers to have a common instruction set despite widely differing costs and speeds. Each machine in the range had an economic balance of instructions executed by microprogram (which was inexpensive but slow) and by hardware (which was faster and but more expensive). Microprogramming remains a fundamental technique of computer design.

本讲提纲

- MIPS指令处理步骤
- 算术逻辑指令数据通路
- 访存指令的数据通路
- 控制流指令数据通路
- MIPS单周期控制逻辑
- MIPS单周期性能分析

单周期微架构分析

- 每条指令需要一个时钟周期来执行
  - CPI (cycles per instruction) 严格为1
- 每条指令需要多久来处理由最慢的指令所需要的处理时间来确定
  - 即使很多指令不需要那么长时间来处理
- 所以，时钟周期由最慢的指令完成执行所需的时间来决定
  - 相应的设计的关键路径相应的也由最慢的指令的处理时间来确定

MIPS中最慢的指令是什么?

- 让我们回到一条指令的处理过程。。。
- 一条指令的5个处理阶段均需要在一个机器周期内完成
  - 取指 (IF)
  - 译码 & 取寄存器操作数 (ID&RF)
  - 执行/计算访存地址 (EX/AG)
  - 访存 (MEM)
  - 回写 (WB)
- 对所有的指令来说，上述5个阶段所需要的时间相等吗?

总 结

- MIPS中指令的处理通常有5个阶段
  - 并不是所有指令均涉及这5个阶段
- 单周期模型中，5个阶段在同一个周期完成
  - 优点是实现简单
  - 缺点是效率不高
- 单周期模型是流水线模型的基础
  - 流水线可以提升指令处理的吞吐率
  - 流水线并不能缩短单个指令的处理时间